QuickCart REST API
Black-Box Testing Report

Roll Number: 2024101116

March 23, 2026

Objective

This document records black-box testing for the QuickCart REST API using only the provided API documentation and externally observable behavior. Internal implementation details were not used.

Test Setup

- Base URL: `http://localhost:8080/api/v1`
- Tools: `pytest`, `requests`
- Test file: `2024101116/blackbox/tests/test_blackbox.py`
- Primary spec source: `QuickCart System.md`
- Required headers:
  - `X-Roll-Number: 2024101116`
  - `X-User-ID: <positive integer>` for user-scoped endpoints

Black-Box Test Design Strategy

Test cases were designed with equivalence partitioning and boundary value analysis, and include:

- Valid requests
- Invalid inputs
- Missing fields
- Wrong data types
- Boundary values

Validation in automation focuses on:

- Correct HTTP status codes
- Proper JSON response structure
- Correctness of returned data against API documentation

Comprehensive Test Case List

ID	Endpoint	Input	Expected Output	Justification
TC-01	GET /admin/users	Missing `X-Roll-Number`	401 Unauthorized	Verifies mandatory global auth header rule.
TC-02	GET /admin/users	`X-Roll-Number=abc`	400 Bad Request	Verifies integer validation for roll header.
TC-03	GET /admin/users	Valid `X-Roll-Number`	200 and JSON list	Baseline valid admin access check.
TC-04	GET /profile	Missing `X-User-ID`	400 Bad Request	Verifies user-scoped header requirement.
TC-05	GET /profile	`X-User-ID=abc`	400 Bad Request	Wrong data type for user id.
TC-06	GET /profile	Valid headers	200 and profile JSON with name/phone fields	Happy path profile retrieval.
TC-07	PUT /profile	name=Test User, phone=9876543210	200 success	Valid profile update.
TC-08	PUT /profile	name length = 1	400	Lower boundary violation for name (min 2).
TC-09	PUT /profile	name length = 51	400	Upper boundary violation for name (max 50).
TC-10	PUT /profile	phone length = 9	400	Lower boundary violation for phone digits.
TC-11	PUT /profile	phone length = 11	400	Upper boundary violation for phone digits.
TC-12	PUT /profile	phone=abcdefghij	400	Wrong type semantics: non-digit phone should fail.
TC-13	GET /addresses	Valid headers	200 and JSON list	Baseline addresses listing.

ID	Endpoint	Input	Expected Output	Justification
TC-14	POST /addresses	label=SHOP	400	Invalid enum value for label.
TC-15	POST /addresses	street length = 2	400	Lower boundary violation for street (min 5).
TC-16	POST /addresses	city length = 1	400	Lower boundary violation for city (min 2).
TC-17	POST /addresses	valid address with pincode=500001	200/201 and created address object	Happy path address creation.
TC-18	POST /addresses	pincode=123	400	Invalid pincode boundary (must be 6 digits).
TC-19	PUT /addresses/id	body=street:new	200 and updated street in response	Valid mutable field update.
TC-20	PUT /addresses/id	body=is_default:true	200 and default switched	Valid default-toggle behavior.
TC-21	PUT /addresses/id	body=label:OFFICE	400	Immutable field protection (label).
TC-22	PUT /addresses/id	body=city:Mumbai	400	Immutable field protection (city).
TC-23	PUT /addresses/id	body=pincode:600001	400	Immutable field protection (pincode).
TC-24	DELETE /addresses/99999	valid headers	404	Non-existent resource handling.
TC-25	GET /products	valid headers	200 and JSON list	Baseline product listing.
TC-26	GET /products	no params	all products active only	Enforces product visibility rule.
TC-27	GET /products/1	valid headers	200 and product_id=1	Baseline product detail retrieval.
TC-28	GET /products/99999	valid headers	404	Non-existent product behavior.
TC-29	GET /products?category=Dairy	valid headers	200 and all results category=Dairy	Filter correctness.
TC-30	GET /products?sort=price_asc	valid headers	prices sorted ascending	Sort correctness asc.
TC-31	GET /products?sort=price_desc	valid headers	prices sorted descending	Sort correctness desc.
TC-32	GET /products?name=Milk	valid headers	only matching names returned	Search behavior correctness.

ID	Endpoint	Input	Expected Output	Justification
TC-33	GET /products + GET /admin/products	compare common product IDs	user-visible prices equal DB prices	Data integrity across views.
TC-34	POST /cart/add	inactive product_id	400/404	Business rule: inactive products not purchasable.
TC-35	DELETE /cart/clear	valid headers	200	Baseline cart reset helper behavior.
TC-36	POST /cart/add	product_id=1, quantity=2	200	Happy path add item.
TC-37	GET /cart	after add	200 and items[] present	Cart retrieval structure validation.
TC-38	POST /cart/add	product_id=99999	404	Non-existent product handling in cart add.
TC-39	POST /cart/add	quantity=99999	400	Stock boundary enforcement.
TC-40	POST /cart/add	quantity=-1	400	Negative quantity rejected.
TC-41	POST /cart/add	quantity=0	400	Boundary check: min quantity is 1.
TC-42	POST /cart/update	existing item quantity=5	200	Valid quantity update.
TC-43	POST /cart/update	quantity=0	400	Boundary check for update endpoint.
TC-44	POST /cart/remove	product not in cart	404	Correct remove error behavior.
TC-45	POST /cart/add twice same product	qty=2 then qty=3	final quantity=5	Merge behavior for duplicate product adds.
TC-46	GET /cart	single item	subtotal = qty * unit_price	Arithmetic correctness item-level.
TC-47	GET /cart	multi-item cart	total = sum(item subtotals)	Arithmetic correctness cart-level.
TC-48	POST /coupon/apply	coupon_code=SAVE50 with eligible cart	200 and discount applied	Valid coupon path.
TC-49	POST /coupon/apply	coupon_code=FOOBAR	400	Invalid coupon handling.
TC-50	POST /coupon/apply	expired coupon (EXPIRED100)	400	Expiry validation.

ID	Endpoint	Input	Expected Output	Justification
TC-51	POST /coupon/apply	PERCENT30 with low cart value	400	Min cart threshold validation.
TC-52	POST /coupon/apply	PERCENT10 on subtotal 600	discount = 60	Percentage discount computation.
TC-53	POST /coupon/apply	PERCENT20 on subtotal 600	discount = 120	Percentage discount computation boundary.
TC-54	POST /coupon/remove	valid headers	200	Coupon removal endpoint behavior.
TC-55	POST /checkout	payment_method=BITCOIN	400	Invalid enum value for method.
TC-56	POST /checkout	payment_method=CARD	200 and payment_status=PAID	Valid CARD flow.
TC-57	POST /checkout	payment_method=COD	200 and payment_status=PENDING	Valid COD flow.
TC-58	POST /checkout	payment_method=WALLET	200 and payment_status=PENDING	Valid WALLET flow.
TC-59	POST /checkout	empty cart	400	Mandatory cart precondition.
TC-60	POST /checkout	COD order total > 5000	400	COD amount rule.
TC-61	POST /checkout	known subtotal	gst_amount = subtotal * 0.05	GST correctness.
TC-62	POST /checkout	known subtotal	total_amount = subtotal + gst_amount	Total formula correctness.
TC-63	GET /wallet	valid headers	200 and wallet_balance field	Baseline wallet response structure.
TC-64	POST /wallet/add	amount=500	200	Valid add money request.
TC-65	POST /wallet/add	amount=0	400	Lower boundary invalid.
TC-66	POST /wallet/add	amount=-100	400	Negative amount invalid.
TC-67	POST /wallet/add	amount=100001	400	Upper boundary invalid.
TC-68	POST /wallet/add	amount=100000	200	Upper boundary valid case.
TC-69	POST /wallet/pay	amount=0	400	Lower boundary invalid for pay.
TC-70	POST /wallet/pay	amount=-10	400	Negative amount invalid.
TC-71	POST /wallet/pay	amount greater than balance	400	Insufficient funds protection.

ID	Endpoint	Input	Expected Output	Justification
TC-72	POST /wallet/pay + GET /wallet	pay amount=50	balance decreases exactly by 50	Monetary precision and correctness.
TC-73	GET /loyalty	valid headers	200 and loyalty_points field	Baseline loyalty structure.
TC-74	POST /loyalty/redeem	points=0	400	Lower boundary invalid.
TC-75	POST /loyalty/redeem	points=-5	400	Negative redeem invalid.
TC-76	POST /loyalty/redeem	points=999999	400	Insufficient points handling.
TC-77	POST /loyalty/redeem	points=1	200	Minimum valid redeem boundary.
TC-78	GET /orders	valid headers	200 and JSON list	Baseline order list retrieval.
TC-79	GET /orders/99999	valid headers	404	Non-existent order detail behavior.
TC-80	POST /orders/99999/cancel	valid headers	404	Non-existent order cancel behavior.
TC-81	GET /orders/99999/invoice	valid headers	404	Non-existent invoice behavior.
TC-82	Checkout then POST /orders/id/cancel	non-delivered order	200	Valid cancellation path.
TC-83	GET /orders/id/invoice	valid order_id	subtotal,gst_amount,total_amount fields present	Invoice response structure.
TC-84	GET /orders/id/invoice	known order	gst_amount=subtotal*0.05	Invoice arithmetic.
TC-85	GET /orders/id/invoice	known order	total_amount=subtotal+gst_amount	Invoice formula correctness.
TC-86	POST /orders/delivered_id/cancel	delivered order	400	Enforce delivered-order cancellation rule.
TC-87	Cancel order flow + product lookup	cancel after placing order	stock restored by cancelled quantity	Inventory consistency on cancel.
TC-88	GET /products/1/reviews	valid headers	200 and average_rating field	Reviews structure baseline.
TC-89	POST /products/1/reviews	rating=5, comment valid	200/201	Valid review creation.
TC-90	POST /products/1/reviews	rating=0	400	Lower boundary invalid rating.
TC-91	POST /products/1/reviews	rating=6	400	Upper boundary invalid rating.

ID	Endpoint	Input	Expected Output	Justification
TC-92	POST /products/1/reviews	rating=-1	400	Negative rating invalid.
TC-93	POST /products/1/reviews	comment=""	400	Empty comment invalid.
TC-94	POST /products/1/reviews	comment length=201	400	Upper boundary invalid comment.
TC-95	POST /products/2/reviews	comment length=200	200/201	Upper boundary valid comment.
TC-96	GET /products/1/reviews	averaged data present	average rating is decimal, not truncated integer	Precision requirement validation.
TC-97	POST /support/ticket	subject valid, message valid	200/201 and status OPEN	Valid ticket creation and default status.
TC-98	POST /support/ticket	subject length=2	400	Lower boundary invalid subject.
TC-99	POST /support/ticket	subject length=5	200/201	Lower boundary valid subject.
TC-100	POST /support/ticket	message=""	400	Empty message invalid.
TC-101	POST /support/ticket	message length=501	400	Upper boundary invalid message.
TC-102	GET /support/tickets	valid headers	200 list	Baseline ticket listing.
TC-103	PUT /support/tickets/id	OPEN to IN_PROGRESS	200	Valid forward transition 1.
TC-104	PUT /support/tickets/id	IN_PROGRESS to CLOSED	200	Valid forward transition 2.
TC-105	PUT /support/tickets/id	CLOSED to OPEN	400	Invalid reverse transition blocked.
TC-106	PUT /support/tickets/id	CLOSED to IN_PROGRESS	400	Invalid reverse transition blocked.
TC-107	POST /support/ticket	exact message with symbols	stored message identical to input	Data integrity of free-text field.

Automation Verification Mapping

The automated test suite in `test_blackbox.py` verifies:

- HTTP status-code expectations for valid and invalid requests
- JSON field presence checks (for example: `order_id`, `gst_amount`, `wallet_balance`, `average_rating`)
- Domain logic checks (cart arithmetic, coupon discount logic, stock restore on cancellation, status transitions)

- Boundary checks and malformed-input checks

Bug Reports (Detailed)

All expected behaviors below are from `QuickCart System.md`. Each bug includes endpoint, full request shape, expected result, and observed actual result.

Bug 1: Valid 6-digit pincode rejected

Endpoint: `POST /api/v1/addresses`
Request Payload:

- Method: POST
- URL: `/api/v1/addresses`
- Headers: `X-Roll-Number: 2024101116`, `X-User-ID: 50`
- Body: `{"label":"HOME","street":"12345 Main Street","city":"Hyderabad","pincode":"500`

Expected: Accept valid 6-digit pincode and create address (200/201).
Actual: 400 with invalid pincode error.

Bug 2: Invalid short pincode accepted

Endpoint: `POST /api/v1/addresses`
Request Payload: same as above but `pincode="123"`.
Expected: 400 (must be exactly 6 digits).
Actual: 200 success.

Bug 3: Address label update allowed

Endpoint: `PUT /api/v1/addresses/1`
Request Payload:

- Method: PUT
- URL: `/api/v1/addresses/1`
- Headers: `X-Roll-Number: 2024101116`, `X-User-ID: 1`
- Body: `{"label":"OFFICE"}`

Expected: 400, because only `street` and `is_default` are mutable.
Actual: 200 accepted.

Bug 4: Address city update allowed

Endpoint: `PUT /api/v1/addresses/1`
Request Payload: body `{"city":"Mumbai"}`.
Expected: 400 (city immutable via update endpoint).
Actual: 200.

Bug 5: Address pincode update allowed

Endpoint: `PUT /api/v1/addresses/1`
Request Payload: body `{"pincode":"600001"}`.
Expected: 400 (pincode immutable via update endpoint).
Actual: 200.

Bug 6: Product prices mismatch admin DB

Endpoint: `GET /api/v1/products` vs `GET /api/v1/admin/products`
Request Payload:

- Method: GET (two calls)
- URL: `/api/v1/products`, `/api/v1/admin/products`
- Headers: user call uses `X-Roll-Number + X-User-ID`, admin uses `X-Roll-Number`
- Body: none

Expected: User-visible price equals real DB price for each product.
Actual: Many mismatches (example: product 8 and 10 values differ).

Bug 7: Product name search does not filter

Endpoint: `GET /api/v1/products?name=Milk`
Request Payload: GET with `name` query parameter.
Expected: Return only matching products.
Actual: Returns full or near-full unfiltered list.

Bug 8: Inactive products can be added to cart

Endpoint: `POST /api/v1/cart/add`
Request Payload: body `{"product_id":<inactive_id>,"quantity":1}`.
Expected: 400/404 for inactive products.
Actual: 200 item added.

Bug 9: Cart accepts quantity 0

Endpoint: `POST /api/v1/cart/add`
Request Payload: body `{"product_id":1,"quantity":0}`.
Expected: 400 because minimum quantity is 1.
Actual: 200 accepted.

Bug 10: Cart subtotal calculation incorrect

Endpoint: `GET /api/v1/cart` after add
Request Payload:

- POST add item: `{"product_id":1,"quantity":2}`
- GET cart

Expected: `subtotal = quantity * unit_price`.
Actual: Subtotal incorrect (example noted as negative).

Bug 11: Cart total incorrect/zero

Endpoint: `GET /api/v1/cart`
Expected: `total = sum(all item subtotals)`.
Actual: total reported as incorrect (often 0 or invalid).

Bug 12: Cart total incorrect with multiple items

Endpoint: `GET /api/v1/cart`
Request Payload: add two distinct items before GET.
Expected: total includes every line item.
Actual: total computation wrong for multi-item cart.

Bug 13: Wrong cart validation message text

Endpoint: `POST /api/v1/cart/add` with negative quantity
Expected: Message should align with docs: quantity must be at least 1.
Actual: Error message indicates `>= 0`.

Bug 14: PERCENT10 coupon uses raw value, not percentage

Endpoint: `POST /api/v1/coupon/apply`
Request Payload: cart subtotal approx 600, body `{"coupon_code":"PERCENT10"}`.
Expected: discount 60 (10% of 600).
Actual: discount 10.

Bug 15: PERCENT20 coupon uses raw value, not percentage

Endpoint: `POST /api/v1/coupon/apply`
Request Payload: subtotal approx 600, body `{"coupon_code":"PERCENT20"}`.
Expected: discount 120 (20% of 600).
Actual: discount 20.

Bug 16: Checkout allows empty cart

Endpoint: `POST /api/v1/checkout`
Request Payload: empty cart, body `{"payment_method":"CARD"}`.
Expected: 400 (cart must not be empty).
Actual: 200 order created (often total 0).

Bug 17: Wallet pay deducts wrong amount

Endpoint: `POST /api/v1/wallet/pay`
Request Payload: after top-up, pay `{"amount":50}`.
Expected: wallet balance decreases exactly by 50.
Actual: observed deduction differs from requested amount.

Bug 18: Delivered order can be cancelled

Endpoint: `POST /api/v1/orders/{order_id}/cancel`
Request Payload: cancel call for an already DELIVERED order.
Expected: 400 (delivered order is non-cancellable).
Actual: 200 cancellation accepted.

Bug 19: Cancelled orders do not restock inventory

Endpoint: `POST /api/v1/orders/{order_id}/cancel`
Request Payload: place order with product quantity, then cancel, then re-read product stock.
Expected: stock restored by cancelled quantity.
Actual: stock remains reduced.

Bug 20: Average rating returned as truncated integer

Endpoint: `GET /api/v1/products/{id}/reviews`
Expected: precise decimal average (for non-integer means).
Actual: average behaves like rounded/truncated integer.

Bug 21: Invalid CLOSED to OPEN transition allowed

Endpoint: `PUT /api/v1/support/tickets/{ticket_id}`
Request Payload: after moving OPEN to IN_PROGRESS to CLOSED, send `{"status":"OPEN"}`.
Expected: 400 (reverse transition not allowed).
Actual: 200 success.

Bug 22: Invalid CLOSED to IN_PROGRESS transition allowed

Endpoint: `PUT /api/v1/support/tickets/{ticket_id}`
Request Payload: from CLOSED, send `{"status":"IN_PROGRESS"}`.

Expected: 400.
Actual: 200 success.

Bug 23: Profile phone accepts letters

Endpoint: `PUT /api/v1/profile`
Request Payload: body `{"name":"Test","phone":"abcdefghij"}`.
Expected: 400 (phone must be exactly 10 digits).
Actual: 200 accepted.

Coverage Summary

- Total designed test cases listed: 107
- Endpoint groups covered: Auth, Profile, Addresses, Products, Cart, Coupons, Checkout, Wallet, Loyalty, Orders, Reviews, Support
- Bug reports documented: 23
- Required black-box categories covered: valid, invalid, missing fields, wrong types, boundary values